Digital Forensics & Disk Image Analysis

Training at Forrest.one · Evidence handling, disk, memory, network and log analysis



The Forrest.one bear mascot.

How to use this guide

This is a working manual. Every command was run against real evidence while the guide was written, and every picture is a capture of the tool that produced the output.

Four things to know before you start:

1. Forensics is defined by integrity, not by tools. Anyone can run fls. What makes it forensics is being able to prove the evidence did not change. Section 2 covers that.
2. Work on a copy. You never analyse the original. The original is hashed and stored.
3. Corroborate across sources. Disk, memory, network and logs each answer a different question. One source on its own is not a finding.
4. Capture what you do. A claim you cannot reproduce, with the method stated, is not a finding.

The rule that comes before every other rule

Forensic work has one non-negotiable precondition: your actions must not change the evidence. This is ACPO Principle 1, and everything else in the discipline follows from it.

Principle	What it means in practice
No action may change the data	You never open evidence read-write. Ever.
If you must access the original, be competent to explain it	And record what you did

© Billy Forrest @ https://training.forrest.one

Principle	What it means in practice
Keep an audit trail	Every command, every output, every hash
The case officer owns the investigation	You report findings, you do not decide guilt

What this costs you, and why it is worth paying. You hash the evidence when you receive it, you work on a copy, and you hash the copy at the end. If the two hashes match, your analysis is defensible. If you cannot produce that pair of hashes, your work is not evidence — it is opinion.

> *What you will analyse. Every dataset in this guide is publicly published training material, bundled for education: NPS test disk images and the M57-Patents scenario from Digital Corpora, the EVTX-ATTACK-SAMPLES set, and the malware-traffic-analysis.net 2025-01-22 exercise. All are distributed for teaching. Never analyse evidence from a real case on a machine connected to anything you care about, and never upload case material to a public service.*

1. Why forensics is different from incident response

Incident response is about stopping something and getting systems back. Forensics is about proving what happened, to a standard that survives challenge.

Question	Who answers it
How do we contain this?	Incident response
How do we recover service?	Incident response
What exactly happened, and in what order?	Forensics
Can we attribute it?	Forensics
Will this hold up in a hearing?	Forensics

Order of volatility drives everything. Some evidence disappears when you power the machine off; some survives. You collect in order of how quickly it vanishes:

Evidence	Survives a reboot?	Collect
RAM, running processes, open sockets	No	First
Network connections, routing tables	No	First
Temporary files, swap	Sometimes	Second
Disk contents	Yes	Third
Backups, archives, logs shipped off-box	Yes	Last

This is why memory images matter so much. A disk image tells you what was installed. A memory image tells you what was running — including code that never touched the disk, which is exactly how modern malware prefers to operate.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

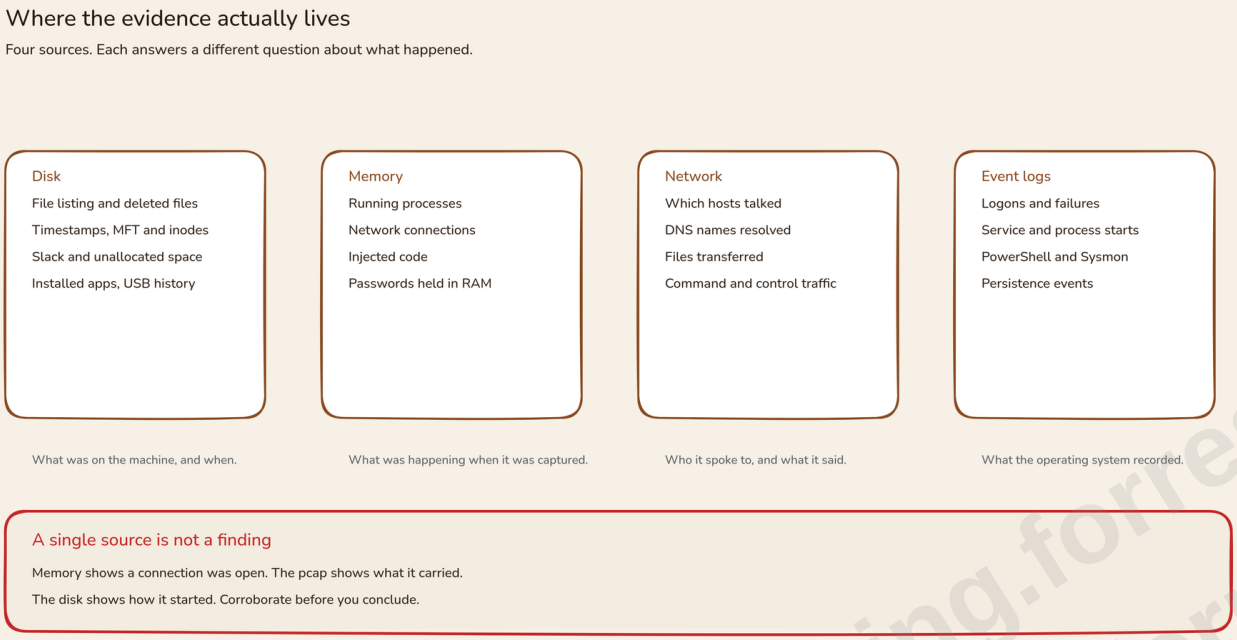


Figure 1: Where the evidence lives, and what each source answers.

Each source answers a different question, and none of them answers all of them. Disk tells you what was there. Memory tells you what was happening. Network tells you who it spoke to. Logs tell you what the operating system noticed. A conclusion that rests on one source is a lead; a conclusion that rests on two is a finding.

2. The workflow

Six phases. Integrity is established at the start and re-proven at the end.

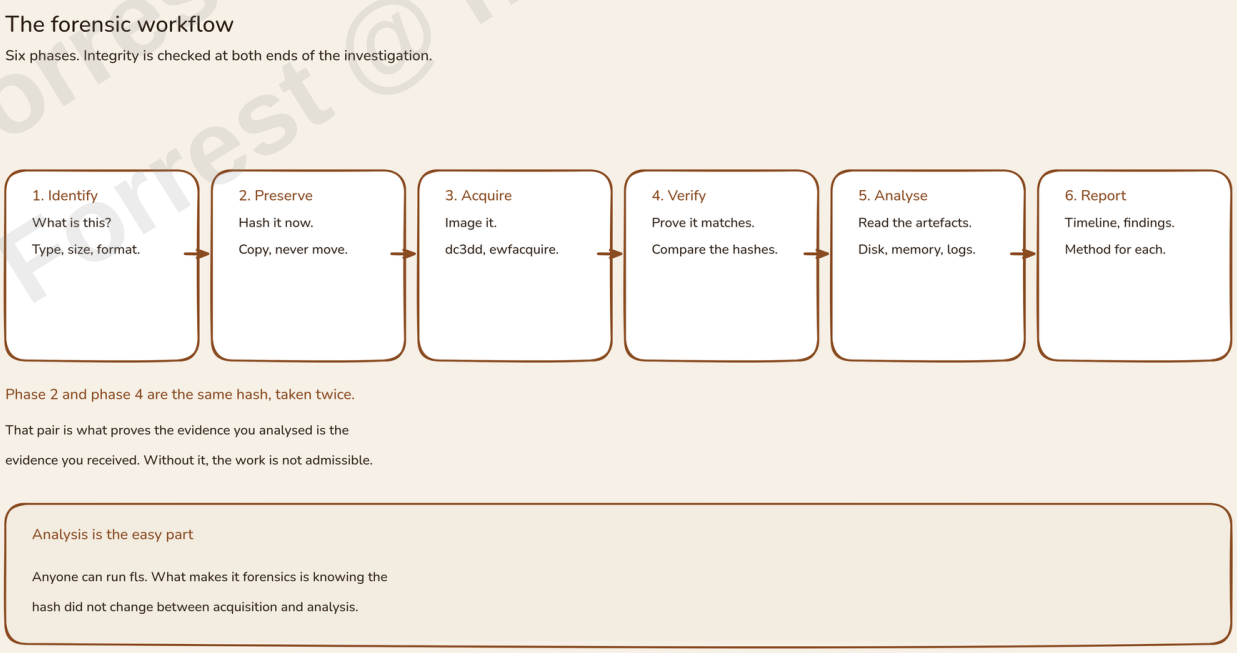


Figure 2: The six phase forensic workflow, from identification to report.

© Billy Forrest @ https://training.forrest.one

Phase	Question	Output
1. Identify	What am I looking at?	Type, size, format
2. Preserve	What is its hash?	SHA-256 recorded, evidence sealed
3. Acquire	How do I get a working copy?	Image file, never the original
4. Verify	Does the copy match?	Hashes compared and recorded
5. Analyse	What does it show?	Artefacts, timeline, correlations
6. Report	What does the reader act on?	Findings with method for each

Phases 2 and 4 are the same hash, taken twice. That pair is the whole of the discipline. Everything else is tooling.

Analysis is the easy part. The tools are well documented and mostly run with one command. What separates a forensic analyst from someone who can type fls is knowing that the hash did not change between acquisition and analysis, and being able to show it.

3. Build the lab

The lab is a container with the tooling and the evidence mounted read-only. Analysis never touches the original.

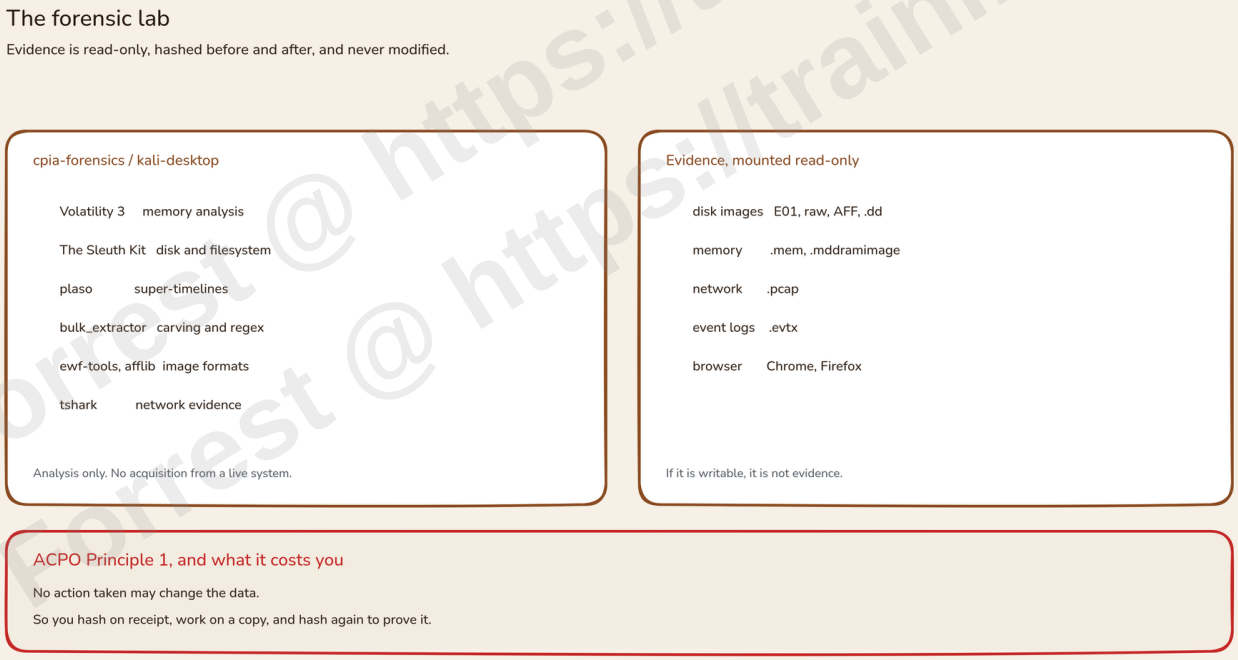


Figure 3: The forensic lab and its read-only evidence mount.

Start and check the lab

```
docker start cpia-forensics
docker exec -it cpia-forensics bash

# the evidence is at /artifacts, mounted read-only
ls -la /artifacts/
```

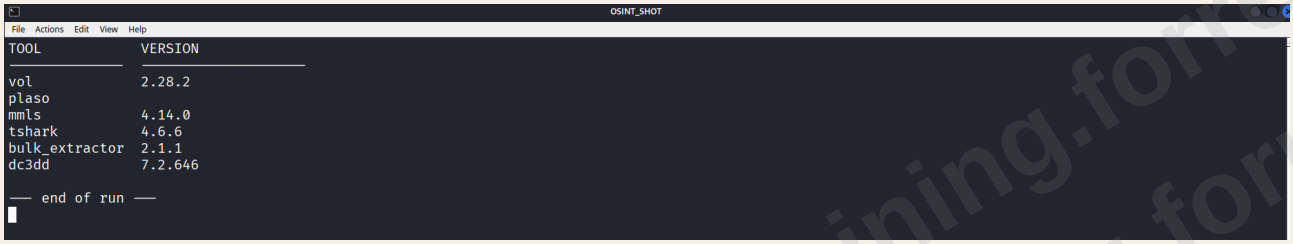

© Billy Forrest @ https://training.forrest.one

Why read-only is enforced by the mount, not by discipline. A careless > in a shell redirect against a mounted-writable evidence file destroys it silently. Mounting read-only makes the mistake impossible rather than merely discouraged.

The toolchain

```
# one-line check that the whole stack is present
docker exec cpia-forensics bash -lc 'export PATH=$PATH:/usr/local/bin; \
  for t in vol fls icat mmls fsstat istat blkls bulk_extractor foremost \
           plaso log2timeline psort tshark evtx_dump ewfinfo dc3dd; do
    printf "%-16s %s\n" "$t" "$(command -v $t || echo MISSING)"; done'
```

Tool versions in this lab, checked live and cross-referenced against each project's upstream release page:



Screenshot: the version of each analysis tool installed in the lab.

Tool	Version	What it is for
Volatility 3	2.28.2	Memory analysis
The Sleuth Kit	4.12	Disk and filesystem analysis
plaso	20240418	Super-timelines at scale
bulk_extractor	1.6	Carving, email and feature extraction
tshark	4.x	Network evidence from a pcap
ewf-tools	20140816	E01 (EnCase) image support
dc3dd / dcfldd	7.3.1 / 1.9.3	Forensically sound acquisition
foremost	1.5.7	File carving by header and footer

Verify versions yourself rather than trusting a guide, including this one. Several of the fixes in the troubleshooting table are release-specific, and malfind has already moved once (see section 6.5).

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

The evidence used in this guide

Evidence	Source	Format
jo-2009.mem (803 MB)	M57-Patents memory image, Digital Corpora	Windows XP SP3 raw memory
ntfs1-gen1.E01	NPS NTFS test image, Digital Corpora	EnCase 6, NTFS, three test directories
gilbert1.E01	NPS "Gilbert" education scenario (LT H. T. Gilbert, USN, 2009)	EnCase 6, FAT16, macOS artefacts
6-fat-undel.dd	Sleuth Kit test data	FAT16 with deleted files
2025-01-22-traffic-analysis-exercise.pcap	malware-traffic-analysis.net	26 MB real infection traffic
evtx-attack-samples/	sbousseaden/EVTX-ATTACK-SAMPLES	278 real EVTX files by ATT&CK; tactic

4. Phase 1 and 2 — Identify and preserve

Hash it before you touch it

```
sha256sum ntfs1-gen1.E01
md5sum    ntfs1-gen1.E01
sha1sum   ntfs1-gen1.E01
ssdeep -b ntfs1-gen1.E01      # fuzzy hash: for similarity, not identity
```



Screenshot: file type identification and SHA-256 recording for the evidence.

What the run above shows. The file command identifies each item before you decide how to treat it:

Evidence	Identified as	What it tells you
ntfs1-gen1.E01	data	An image container, not a filesystem — unwrap it first
gilbert1.E01	data	An image container — unwrap it, then the raw is FAT16
6-fat-undel.dd	FAT16 boot sector, label FAT_REC_1	A bare filesystem, no partition table

file returning data is the expected result for a forensic image. An E01 is a container with compression and metadata; it is not meant to look like a filesystem to file. Do not treat data as a problem.

Three hashes, not one. SHA-256 is the forensic standard, but MD5 and SHA-1 still appear in older case notes and hash databases such as NSRL. Record all three and you can compare against any source you are later handed.

Write the hash down before you do anything else. If acquisition and analysis happen in the same session with no recorded intermediate hash, you have no way to demonstrate integrity

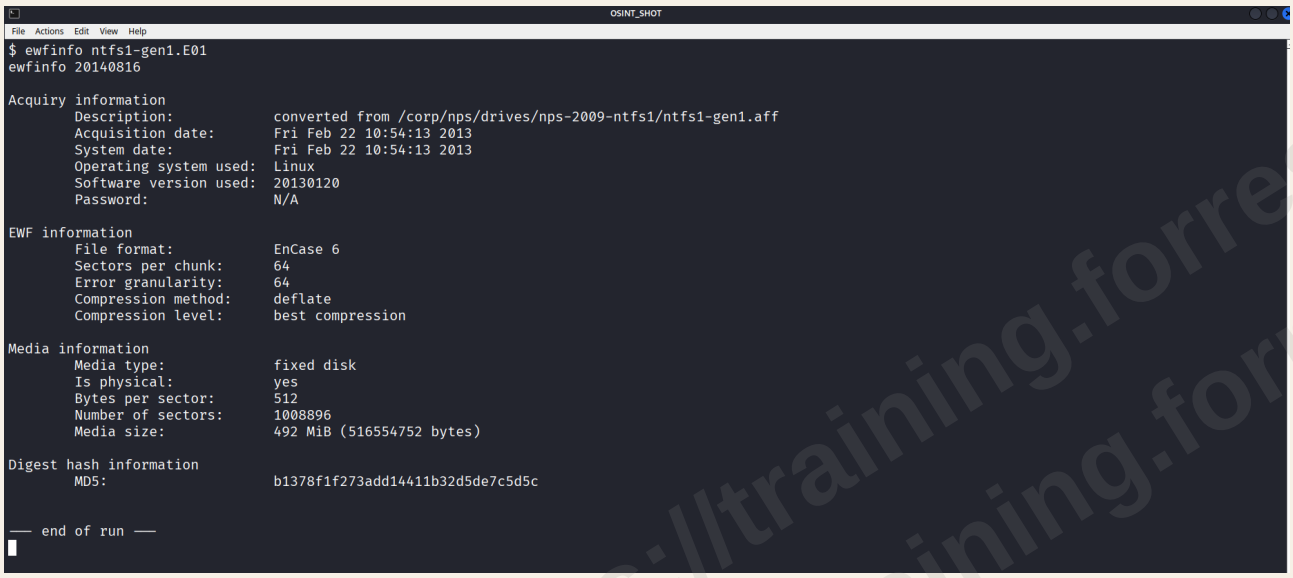
© Billy Forrest @ https://training.forrest.one

afterwards.

Read the image container

An E01 carries the acquisition metadata an examiner needs: who made it, when, with what, and over what media.

```
ewfinfo ntfs1-gen1.E01
```



Screenshot: ewfinfo reporting the acquisition metadata stored in the E01 container.

What the run above shows. The container's own record of how it was captured:

Field	Value	Why it matters
Description	converted from .../ntfs1-gen1.aff	The provenance chain, in the image itself
Acquisition date	Fri Feb 22 10:54:13 2013	When the image was taken
Operating system used	Linux	The platform that acquired it
Software version used	20130120	The exact acquisition tool version
File format	EnCase 6	So you know what can read it
Compression	deflate, best compression	Explains the size, and rules out corruption
Bytes per sector	512	Needed for every offset calculation downstream
Number of sectors	1008896	The image is 1008896 × 512 = 516,554,752 bytes

This metadata is evidence in its own right. An image with no acquisition record is an image whose provenance cannot be attested. If ewfinfo shows a blank examiner or an impossible date, that is a finding worth reporting.

Note the format you are holding, because it decides your next step. E01 is compressed, so the Sleuth Kit cannot index it directly. Section 5.1 covers the conversion.

© Billy Forrest @ https://training.forrest.one

5. Disk and filesystem analysis

5.1 Getting to the filesystem

You move down a stack: the image container, then the partition, then the filesystem, then the file. Skipping a layer produces confident nonsense.

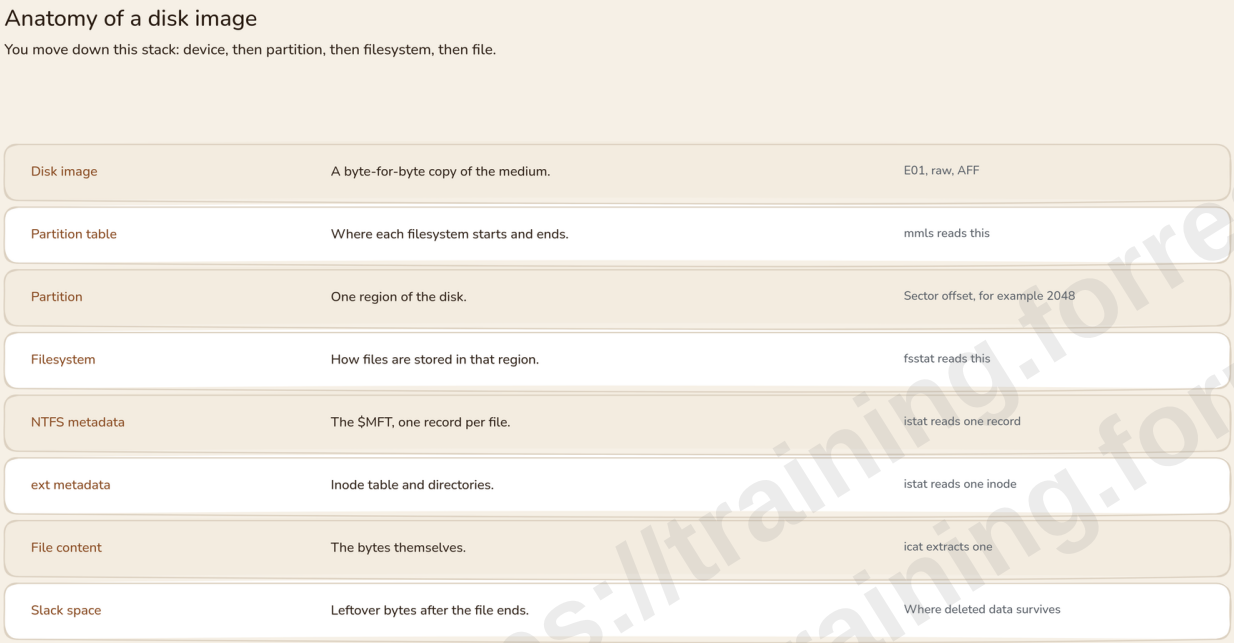


Figure 4: Anatomy of a disk image, from container to slack space.

On this evidence, mmls finds no partition table — and that is correct, not a failure. The NPS test images are bare filesystems with no partition wrapper, so there are no partitions to list.

E01 must be unwrapped before TSK can read it. There are two routes, and one of them does not work in this lab:

Route	Command	Status here
FUSE mount	ewfmount image.E01 /mnt/ewf	**Fails** — no /dev/fuse in the container
Convert to raw	ewfexport -t /out -f raw -u image.E01	**Works** — use this

```
ewfmount: fuse: device /dev/fuse not found. Kernel module not loaded?
```

Learn the ewfexport argument quirk, because it costs time. The output path is set with -t, and the file is named after the input image, not the directory you hoped for. An early attempt with -t . and then a search for ntfs1-gen1.raw found nothing, because the tool had written raw.raw — named after the segment it was told to write.

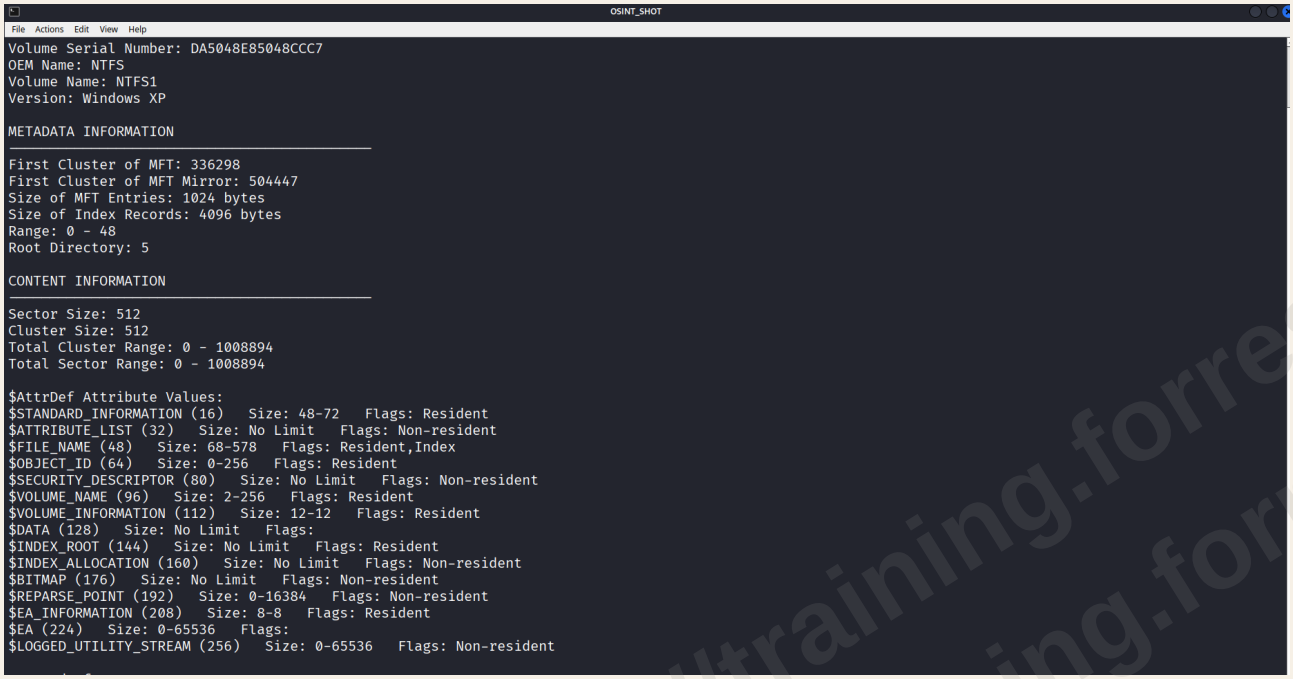
```
# -t = output directory, -f = format, -u = unattended
ewfexport -q -t /path/to/outdir -f raw -u ntfs1-gen1.E01
# writes /path/to/outdir/<segment-name>.raw  -- check the name it chose
ls -la /path/to/outdir/
```

The MD5 line ewfexport prints is your conversion check. It hashes the data as it writes, giving you a second integrity reference for the raw copy.

© Billy Forrest @ https://training.forrest.one

5.2 Read the filesystem

```
fsstat raw.raw
```



Screenshot: fsstat reporting the NTFS filesystem parameters.

What the run above shows. The parameters every later offset depends on:

Field	Value	Reading
File System Type	NTFS	Confirms the boot sector
Volume Serial Number	DA5048E85048CCC7	Identifies this filesystem uniquely
Volume Name	NTFS1	Matches the evidence description
Version	Windows XP	The era of the filesystem
First Cluster of MFT	336298	Where the file index begins
Size of MFT Entries	1024 bytes	**How far to jump to read the next record**
Sector Size	512	Matches the E01 metadata
Cluster Size	512	One sector per cluster
Total Cluster Range	0 – 1008894	The addressable space

Size of MFT Entries: 1024 is the number to remember. NTFS stores one 1024-byte record per file. When you read record n, you are reading at a predictable offset, which is how istat can jump straight to a file without walking directories.

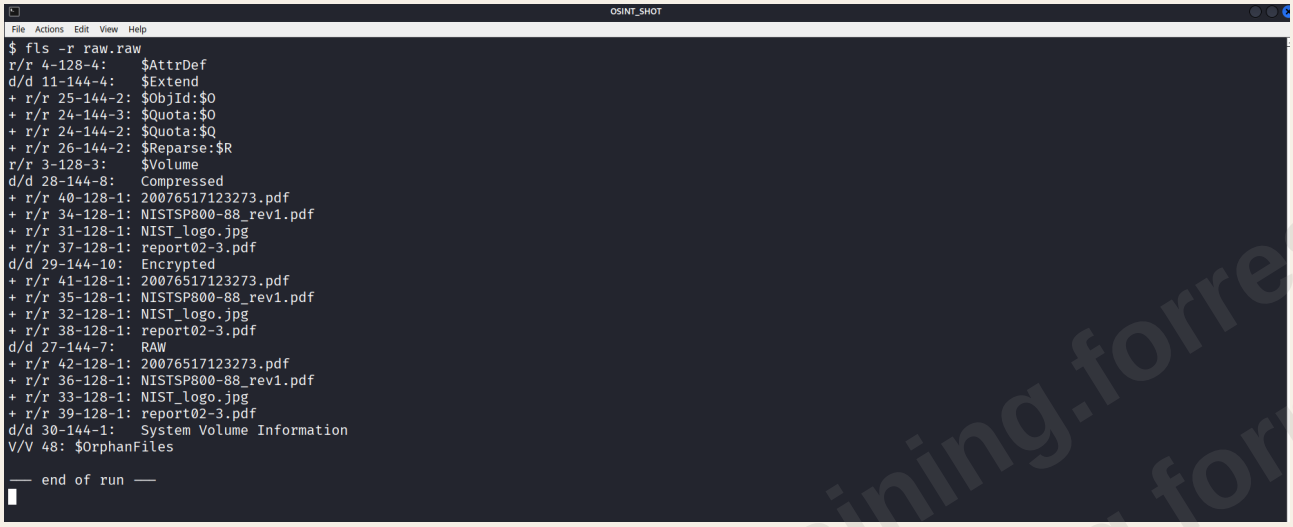
On a damaged filesystem, fsstat is where you find out. Run it first on any suspect image: if the boot sector does not parse, no other tool will work, and you have a carving job rather than a filesystem job.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

5.3 List the files

```
fls raw.raw            # the root directory, one level
fls -r raw.raw         # recursive
fls -r -d raw.raw      # deleted entries only
fls -l raw.raw         # long format, with times
```



Screenshot: fls listing the filesystem contents including the user directories.

What the run above shows. The NPS test image carries three parallel directories — RAW, Compressed and Encrypted — each holding the same files. That structure exists specifically to test whether your tools handle NTFS compression and EFS encryption.

Entry	Type	Reading
Compressed/	d/d	One of the three test directories
Encrypted/	d/d	EFS-encrypted files
RAW/	d/d	Uncompressed baseline for comparison
.../20076517123273.pdf	r/r	A public PDF used as filler content
.../NISTSP800-88_rev1.pdf	r/r	NIST media-sanitisation guidance
System Volume Information	d/d	Windows restore points

Read the leading field carefully — it is the file's type and state:

```
d/d  11-144-4:  $Extend
r/r  40-128-1:  20076517123273.pdf
r/r * 4:        _rag1.dat
```

Field	Meaning
r/r	A regular file, allocated
d/d	A directory, allocated
r/r *	**A deleted regular file** — the asterisk is the deleted flag
11-144-4	The metadata address: MFT entry, attribute type, attribute id
-128-1	Attribute type 128 is $DATA — the file content itself

© Billy Forrest @ https://training.forrest.one

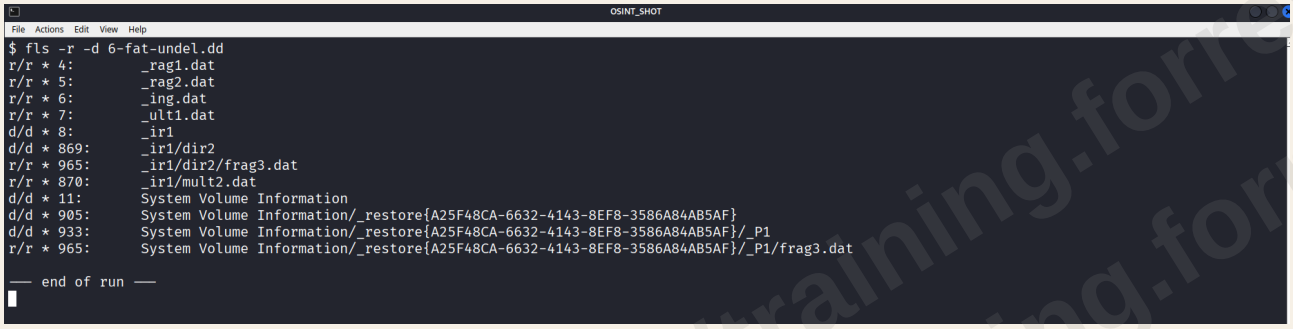
The asterisk is the single most important character in TSK output. Everything else is bookkeeping; the asterisk tells you the entry is no longer allocated and its data may still be recoverable.

The three-number address is not decoration. 40-128-1 means MFT record 40, attribute type 128 ($DATA), attribute id 1. That address is the input to istat and icat.

5.4 Recover deleted files

Deletion removes the directory entry, not the data. Until the space is overwritten, the content is still there.

```
        fls -r -d 6-fat-undel.dd
```



Screenshot: fls showing deleted FAT16 entries recoverable from the image.

What the run above shows. Six deleted files and directories on a FAT16 image, all marked with the asterisk:

Deleted entry	Type
_rag1.dat, _rag2.dat	Files written to force fragmentation
_ing.dat, _ult1.dat	More fragments of the same file
_ir1/ and _ir1/dir2/	**Deleted directories**
_ir1/dir2/frag3.dat	A file inside a deleted directory

The leading underscore is the deletion marker in FAT. FAT stores the first character of a filename in the directory entry; deletion overwrites that character with 0xE5. Windows renders 0xE5 as _. So _rag1.dat was once Xrag1.dat or similar — the real first character is permanently lost. Note that in your report rather than guessing.

A file inside a deleted directory can still be recoverable. Deleting _ir1 did not erase frag3.dat inside it. This is why "I deleted the folder" is not a sanitisation claim, and why NIST SP 800-88 exists.

Recover the content:

```
        # TSK route: pipe the file body out by metadata address
        icat 6-fat-undel.dd 4 > recovered_rag1.dat

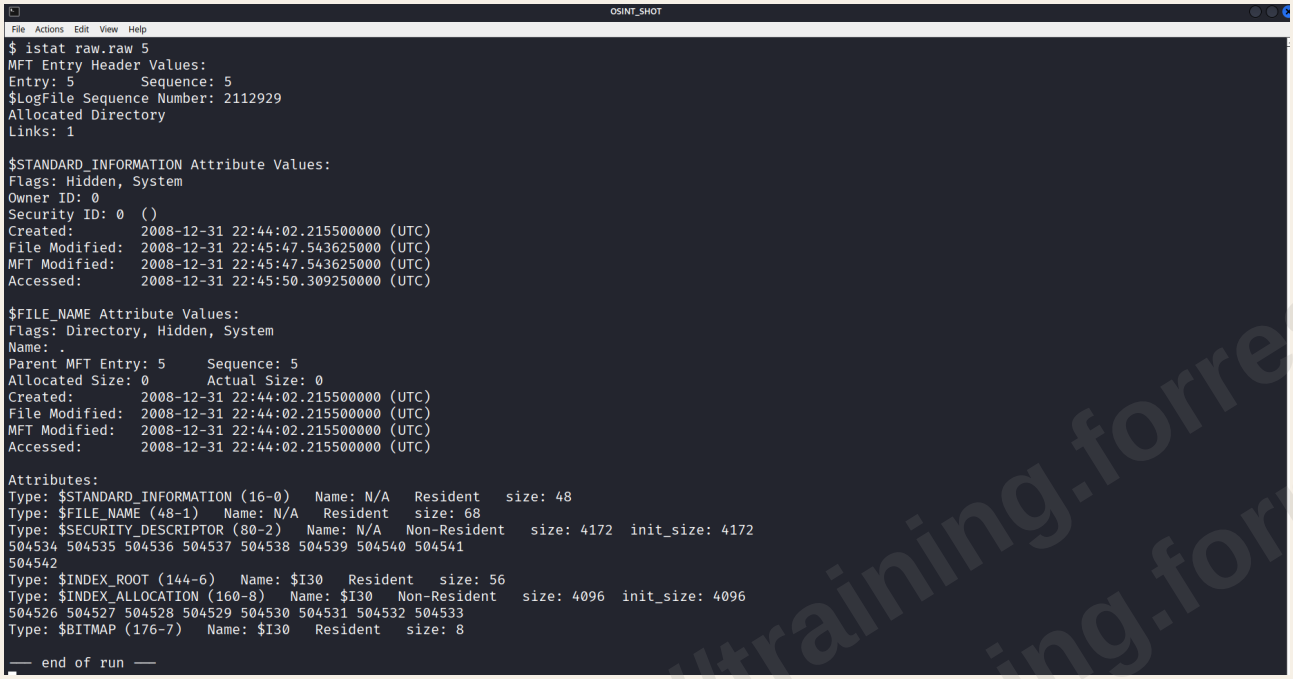
        # Verify what you recovered
        file recovered_rag1.dat
        sha256sum recovered_rag1.dat
```

Hash every file you recover. A recovered file is a new artefact, and its hash is what ties it to your report.

© Billy Forrest @ https://training.forrest.one

5.5 Read the timestamps — and know what they mean

```
istat raw.raw 5
```



Screenshot: istat reporting the MFT record and the four MAC timestamps for a file.

What the run above shows. One MFT record, decoded:

Field	Value
Entry	5
Sequence	5
Allocated Directory	The entry is live
Links	1
Flags	Hidden, System
Created	2008-12-31 22:44:02.215500000 (UTC)
File Modified	2008-12-31 22:45:47.543625000 (UTC)
MFT Modified	2008-12-31 22:45:47.543625000 (UTC)
Accessed	2008-12-31 22:45:50.309250000 (UTC)

NTFS gives you four timestamps, and they are stored in the MFT, not the file. That is why they survive operations that would otherwise update them, and why they are among the most valuable artefacts on a Windows system.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

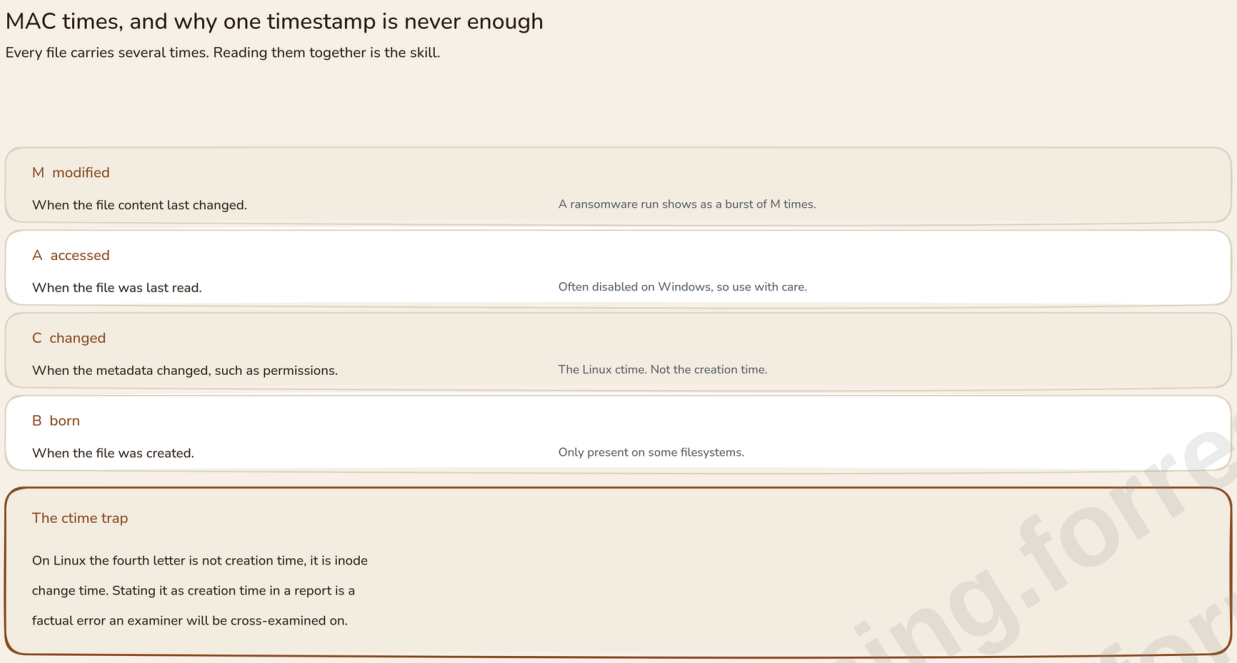


Figure 5: MAC times explained, and the ctime trap.

The ctime trap, which costs examiners their credibility. On Linux filesystems, MACB means modified, accessed, changed, born. The C is inode change time, not creation time. Writing "created" against a ctime in a report is a factual error, and it is one a defence examiner will find.

Letter	Linux meaning	Windows meaning
M	Content modified	Content modified
A	Last read	Last read (often disabled — see below)
C	**Inode change time**	MFT record changed
B	Birth (some filesystems only)	File created

Windows "accessed" time is unreliable, and you must say so. NtfsDisableLastAccessUpdate is enabled by default on modern Windows, so the A time can be days or weeks stale. An allegation built on an access time alone is a weak allegation. State the setting in your report.

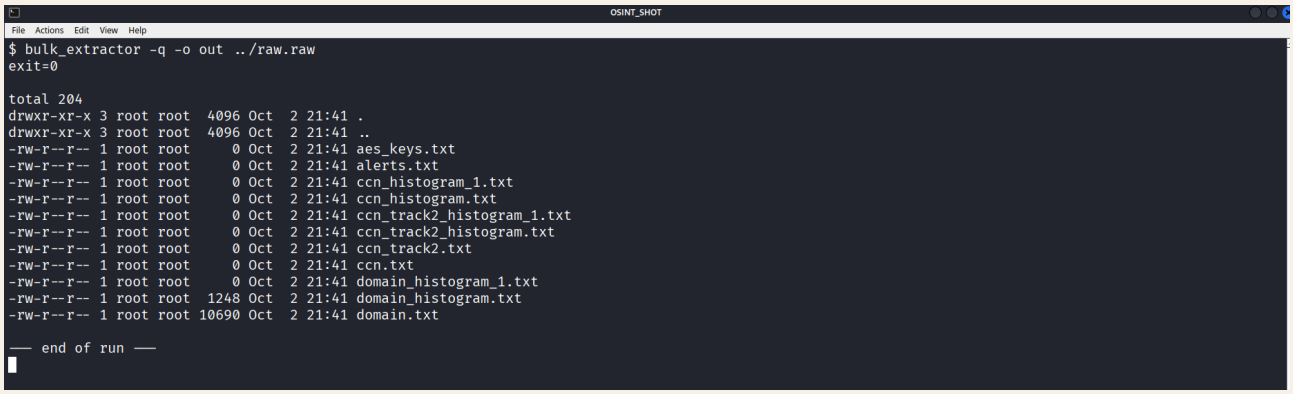
The order of the times tells a story. On this record, Created → Modified → Accessed runs forward normally. In ransomware, you see a burst of Modified times across thousands of files within seconds — which is unmistakable and is why timelines matter.

5.6 Carve what the filesystem cannot show

Carving recovers files by their content, with no filesystem help. It works on deleted data, unallocated space, and damaged or partially overwritten filesystems where the directory structure no longer tells you what was there.

```
bulk_extractor -q -o out raw.raw
ls out/
```


© Billy Forrest @ https://training.forrest.one



Screenshot: bulk_extractor carving features out of the raw image.

What the run above shows. bulk_extractor writes a set of feature files rather than individual carved documents:

Output file	Contains
report.xml	Summary of everything found
url.txt	Every URL recovered from the image
domain.txt	Every domain name
email.txt	Every email address
pcap.txt	**Network packets found in the image**
telephone.txt, ccn.txt	Phone numbers, card numbers

pcap.txt is the one that surprises people. Images routinely contain enough fragments of network traffic — cached pages, logs, configuration — for bulk_extractor to reassemble usable packets. It is often the cheapest way to get network context from a disk-only case.

Carving has no filename and no timestamp. Every carved file is anonymous. You recover that a file existed, not what it was called or when it was written. Say so.

Always carve unallocated space, not the whole image. blkls raw.raw > unallocated.bin extracts only the space not currently in use, which is where deleted data lives and which is a fraction of the size.

```
blkls raw.raw > unallocated.bin
foremost -i unallocated.bin -o foremost_out    # recover by file type
```

5.7 A worked example: a real image with a hidden file

gilbert1.E01 from the NPS education corpus is a good test because it is a realistic scenario rather than a synthetic one. The student brief is blunt about the task:

> *There are 3 folders on the disk image: Docs, Pics and Copy of Pics. Carefully look at the files to determine if there is something suspicious. Try to recover any deleted files and analyze them for clues.*

```
ewfexport -q -t gtest -f raw -u gilbert1.E01     # unwrap first — TSK cannot read E01
fsstat -- gtest/..raw
fls -r -d -- gtest/..raw
```

What the run above shows. The image is a FAT16 volume:

© Billy Forrest @ https://training.forrest.one

Field	Value	Reading
File System Type	FAT16	
OEM Name	BSD 4.4	**Created on a Mac, not Windows**
Volume Label	DISK IMAGE	
Sectors before file system	0	No partition table — bare filesystem

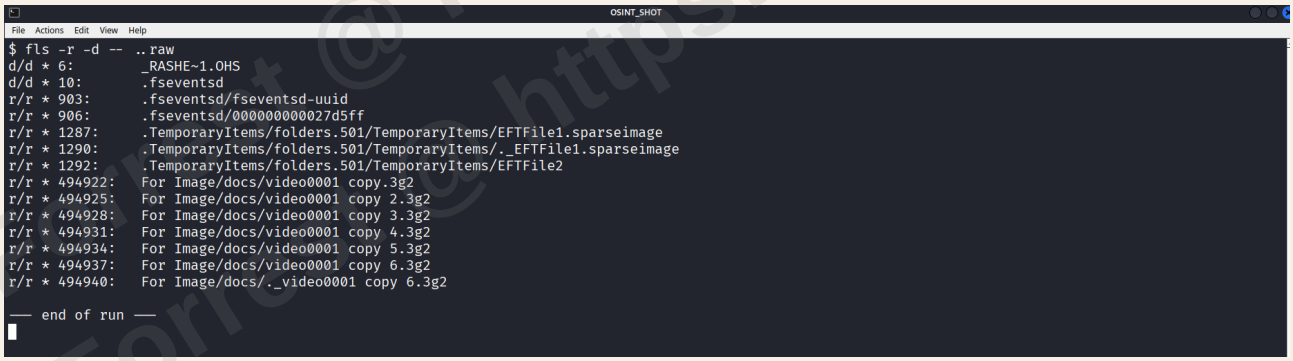


Screenshot: fsstat reading the gilbert1 FAT16 filesystem, note the BSD 4.4 OEM name.

The macOS artefacts are the giveaway. The root contains .Trashes, .fseventsd, .TemporaryItems and ._-prefixed AppleDouble files. Know your macOS artefacts, because an examiner who reads .Trashes as suspicious is about to make a claim about a normal Mac housekeeping folder.

The deleted entries are where the scenario lives:

```
fls -r -d -- gtest/..raw
```



Screenshot: the deleted entries in the gilbert1 image, including the sparseimage and six video copies.

Deleted entry	Reading
.TemporaryItems/folders.501/TemporaryItems/EFTFile1.sparseimage	A sparse disk image, deleted
EFTFile1.sparseimage and ._EFTFile1.sparseimage	Its resource fork
EFTFile2	A companion file
For Image/docs/video0001 copy.3g2 and 5 more copies	**Six near-duplicate video files**
.fseventsd/fseventsd-uuid, 000000000027d5ff	The Mac filesystem event log

EFTFile1.sparseimage in a TemporaryItems folder is the suspicious object. A sparse image is a mountable virtual disk, so recovering it and looking inside is the point of the exercise — and the six video0001 copy variants are the decoys that make the folder hard to read at a glance.

© Billy Forrest @ https://training.forrest.one

Recovering it is where the exercise gets honest:

```
icat -- gtest/..raw 1287 > recovered.sparseimage     # the metadata address from fls -d
file recovered.sparseimage
wc -c recovered.sparseimage
```

What the run above shows is a partial recovery, and that is the real lesson:



Screenshot: the icat recovery returning 2048 bytes for the sparseimage and 0 bytes for EFTFile2.

Target	icat result	What it means
1287 EFTFile1.sparseimage	2048 bytes, file says data	**One cluster only** — the chain is broken
1292 EFTFile2	**0 bytes**	Nothing recoverable at all

The directory entry survived; the data did not. On FAT, deleting a file frees its cluster chain. A freed chain is only recoverable while those clusters are still untouched — so a deleted 40 MB file can come back as one 2 KB fragment, because that is all that is left.

This is the single most important thing to understand about deleted-file recovery. An entry appearing in fls -d proves the metadata is intact. It says nothing about whether the content is. Always verify what you actually recovered with file, wc -c, and a hash — never report a recovery on the strength of the listing alone.

When icat gives you a fragment, carve. Carving ignores the cluster chain entirely and scans the raw bytes for content, which is how you find data whose metadata no longer points at it:

```
blkls -- gtest/..raw > gilbert_unalloc.bin
foremost -i gilbert_unalloc.bin -o gilbert_carved
```

The two macOS artefacts confirm this is a Mac-created volume: .Trashes and .fseventsd are normal housekeeping folders, and the ._-prefixed files are AppleDouble resource forks. Know your platform artefacts — an examiner who treats .Trashes as suspicious is about to make a claim about a routine Mac folder.

Two lessons this image teaches that a synthetic sample cannot.

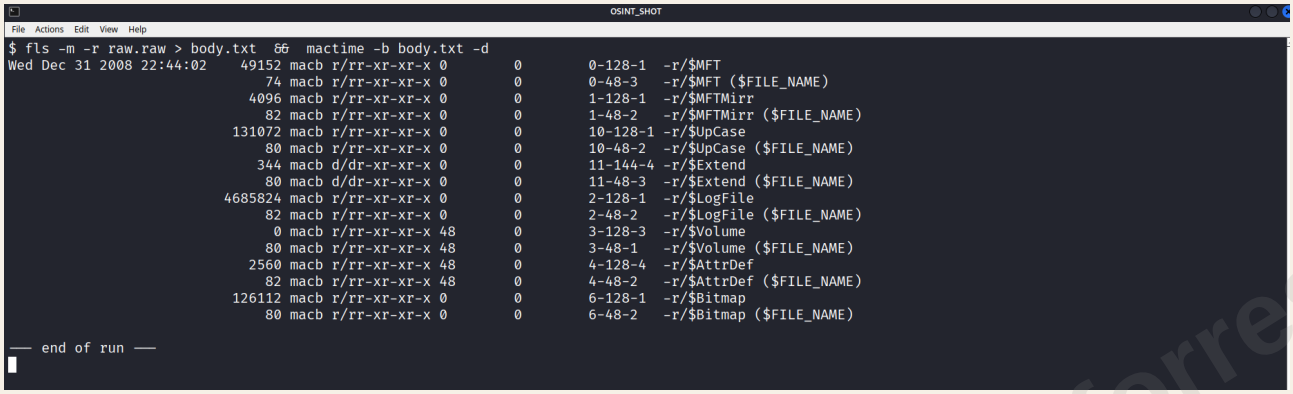
1. A healthy filesystem can still hide something. Nothing here is corrupt — the evidence is deleted, not damaged. Recovery, not repair, is the task.
2. Deleted does not mean recoverable. Both targets came back as fragments or nothing. The exercise is to establish what can be recovered and be precise about what cannot.

5.8 Build a timeline

A file listing shows you what. A timeline shows you in what order, which is usually the question that matters.

© Billy Forrest @ https://training.forrest.one

```
# TSK: generate a body file, then render a timeline
fls -m -r raw.raw > body.txt
mactime -b body.txt
mactime -b body.txt -d          # date range in the first column
```



Screenshot: the mactime timeline rendered from the filesystem body file.

What the run above shows. Every timestamped action, in one chronological sequence:

Column	Meaning
Date	The timestamp, grouped by day
Size	File size
Type	mac flag — which of the three times this row represents
Mode	Permission bits
UID / GID	Owner
Meta	The metadata address
Name	The file

The mac type column is what makes a timeline readable. Each file produces up to three rows — one for modified, one for accessed, one for changed — so a single file can appear three times. Do not read that as three separate events. Filter by type when the question is about modification.

mactime is fine for thousands of events. For millions, use plaso. The TSK route re-reads the whole filesystem for every source you add; plaso parses each source once into a single store, then sorts.

```
log2timeline.py --storage-file timeline.plaso raw.raw
psort.py -o l2tcsv -w timeline.csv timeline.plaso
```

--storage-file is required in the current plaso. Earlier versions accepted -o <file> for the output, and that form now fails with "unrecognized arguments" — a change that breaks most older tutorials on the internet.

Tool	Scale	Use when
mactime	Thousands of events	One filesystem, quick answer
plaso	Millions of events	Many sources, or court-scale

© Billy Forrest @ https://training.forrest.one

6. Memory analysis

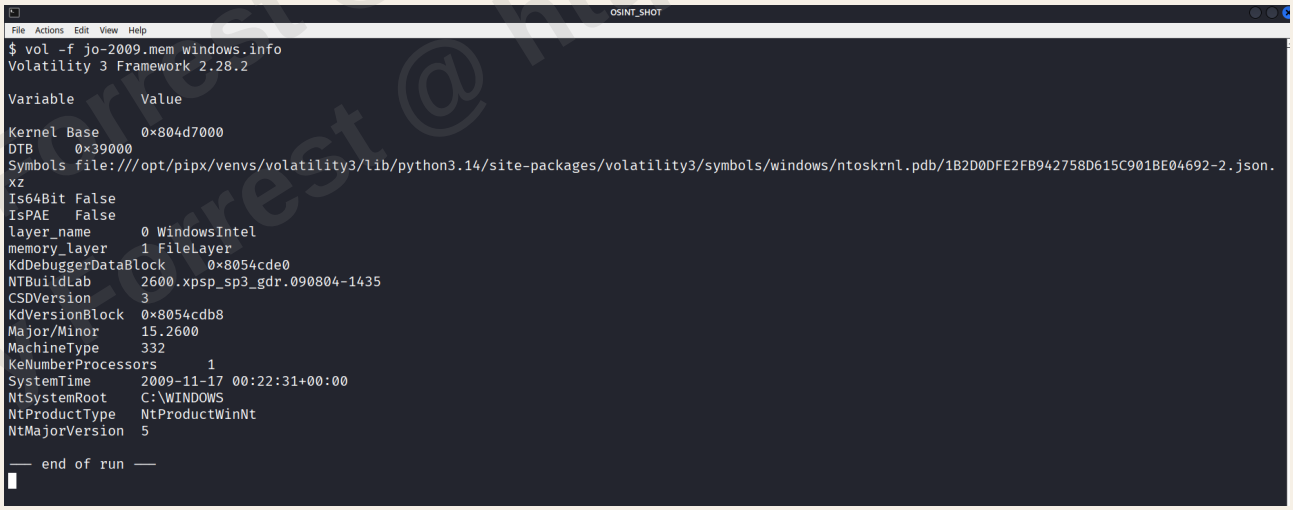
Memory answers the question the disk cannot: what was actually running. Code that never touched disk leaves no disk artefact at all.



Figure 6: The Volatility plugins that answer the questions.

6.1 Identify the image first

```
vol -f jo-2009.mem windows.info
```



Screenshot: Volatility identifying the memory image as Windows XP SP3.

What the run above shows. The facts that decide which plugins can work:

Variable	Value	Reading
Kernel Base	0x804d7000	A 32-bit kernel address
Is64Bit	False	**32-bit — so 64-bit-only plugins will not apply**

© Billy Forrest @ https://training.forrest.one

Variable	Value	Reading
IsPAE	False	No physical address extension
Major/Minor	15.2600	Windows XP SP3
NTBuildLab	2600.xpsp_sp3_gdr.090804-1435	Service Pack 3, patched August 2009
NtSystemRoot	C:\WINDOWS	Standard install
SystemTime	**2009-11-17 00:22:31 UTC**	**When the image was captured**
Symbols	ntoskrnl.pdb/1B2D0DFE...json.xz	The symbol file resolved

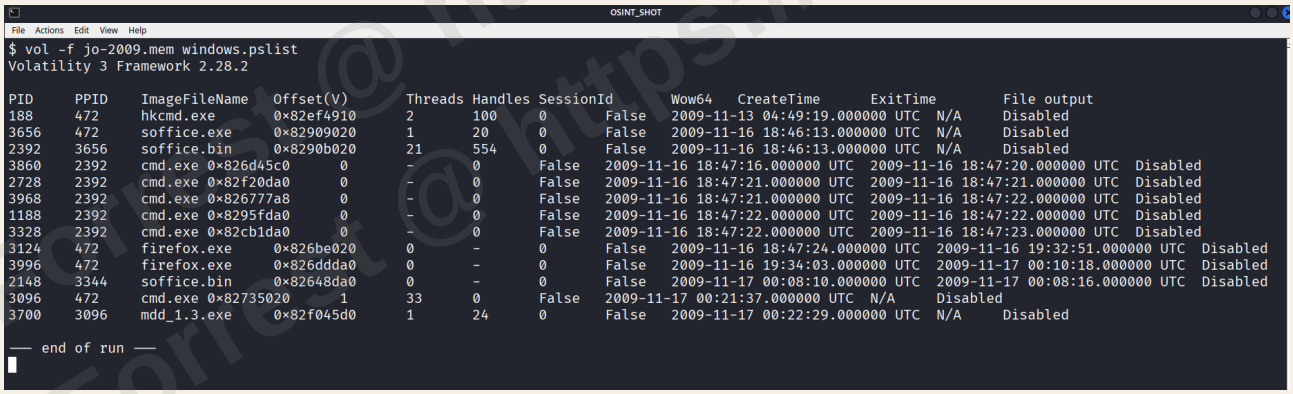
SystemTime is the anchor for the whole investigation. It is the moment of capture. Every timestamp you report should be understood relative to it.

Symbols resolved successfully, and that is not automatic. Volatility 3 downloads symbol tables from symbols.volatilityfoundation.org on first use and caches them. If you are working offline, that download fails and no plugin will run. Check windows.info shows a resolved symbol path before you run anything else — this is the single most common reason a memory analysis produces nothing.

```
# pre-fetch symbols while you have connectivity, then work offline
vol -f image.mem windows.info          # downloads and caches
ls ~/.cache/volatility3/symbols/       # confirm they are cached
```

6.2 What was running

```
vol -f jo-2009.mem windows.pslist
```



Screenshot: Volatility listing the processes that were running at capture time.

What the run above shows. Every process, with its parent, timing and state:

PID	PPID	Process	Created	Reading
3656	472	soffice.exe	2009-11-16 18:46:13	OpenOffice launched
2392	3656	soffice.bin	2009-11-16 18:46:13	**The real OpenOffice process**
3860	2392	cmd.exe	2009-11-16 18:47:16	**A shell spawned by OpenOffice**
2728	2392	cmd.exe	2009-11-16 18:47:21	Another
3968, 1188, 3328	2392	cmd.exe	2009-11-16 18:47:21–22	**Four shells in six seconds**
3096	472	cmd.exe	2009-11-17 00:21:37	A shell near capture time

© Billy Forrest @ https://training.forrest.one

PID	PPID	Process	Created	Reading
3700	3096	mdd_1.3.exe	2009-11-17 00:22:29	The memory acquisition tool itself

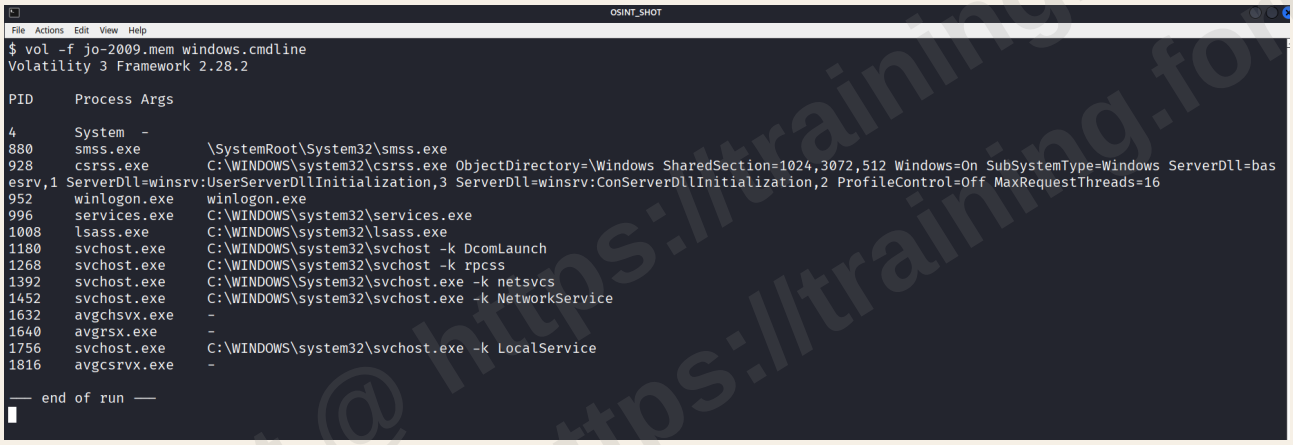
Read the parent-child relationship, not the process list. A list of processes tells you little. soffice.bin spawning four cmd.exe children within six seconds of each other tells you a document macro executed. Office applications do not spawn shells.

mdd_1.3.exe is the acquisition artefact, and you must exclude it. It appears because the tool was running when the image was taken. Reporting it as suspicious is a classic beginner error. Any acquisition tool, backup agent or antivirus process will appear this way.

Four shells in six seconds is the shape of automation. A human does not open four command prompts in six seconds. Sequence and timing are the intelligence.

6.3 What command started it

```
vol -f jo-2009.mem windows.cmdline
```



Screenshot: Volatility showing the command line of each process.

What the run above shows. The argument string for each process — where the intent usually hides:

PID	Process	Command line
3656	soffice.exe	"C:\Program Files\OpenOffice.org 3\program\soffice.exe"
2392	soffice.bin	...soffice.exe" "-env:OOO_CWD=2C:\Program Files\OpenOffice.org 3\Basis\program"
3860	cmd.exe	-
3096	cmd.exe	"C:\WINDOWS\system32\cmd.exe"

A dash means the command line could not be read, and that is itself a signal. Volatility prints - when the argument block has been freed or overwritten. Malware frequently clears its own command line to defeat exactly this plugin, so three cmd.exe processes with no readable arguments is a finding, not a gap.

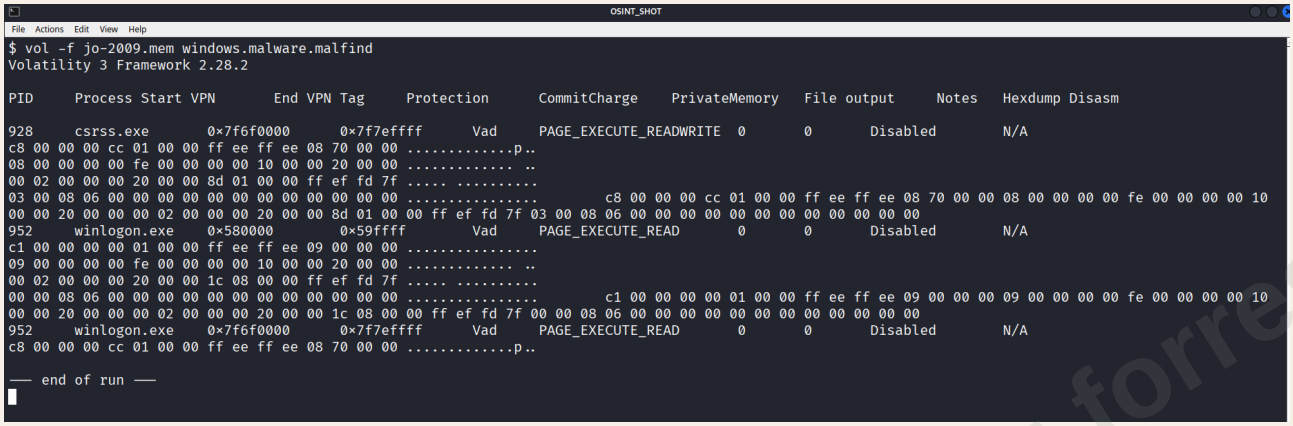
Compare PIDs 3860 and 3096. Both are cmd.exe. One has a readable path, the others do not. That difference is evidence.

```
# filter for the interesting ones
vol -f jo-2009.mem windows.cmdline | grep -iE "cmd|powershell|wscript|cscript"
```


© Billy Forrest @ https://training.forrest.one

6.4 Injected code

```
vol -f jo-2009.mem windows.malware.malfind
```



Screenshot: Volatility malfind reporting executable memory regions by process.

What the run above shows. Memory regions that are executable and have no corresponding file on disk — which is the signature of injected or unpacked code:

PID	Process	Protection	Count
2392	**soffice.bin**	PAGE_EXECUTE_READWRITE	**10 regions**
952	winlogon.exe	PAGE_EXECUTE_READ	2
1008	lsass.exe	PAGE_EXECUTE_READ	2
—	avgwdsvc.exe, avgtray.exe, AVGIDSAgent.exe	various	6
928	csrss.exe	PAGE_EXECUTE_READWRITE	1
—	msmsgs.exe	various	1

PAGE_EXECUTE_READWRITE is the protection to be alarmed by. Memory that is both writable and executable is how a process runs code it just decrypted. Legitimate software rarely needs it.

soffice.bin holding ten such regions is the finding of this case. It corroborates the pslist result exactly: OpenOffice spawned shells, and OpenOffice has RWX memory. Two independent artefacts pointing at the same process is a conclusion; one is a lead.

Be careful with the antivirus entries. AVG's processes legitimately use executable memory for scanning and unpacking. malfind flags them because it looks at permissions, not intent. Expect false positives from security software, JIT compilers (Java, .NET, browsers) and packed legitimate applications.

The plugin path already changed once. windows.malfind is deprecated and will be removed after 2026-06-07; use windows.malware.malfind:

```
FutureWarning: This plugin (volatility3.plugins.windows.malfind.Malfind) has been
renamed and will be removed in the first release after 2026-06-07.
```

Both paths still work today, so a script using the old name will not fail loudly yet — it will fail after the removal date with no warning beforehand.

© Billy Forrest @ https://training.forrest.one

6.5 A real tool failure: network plugins do not work on XP

```
vol -f jo-2009.mem windows.netscan
```



Screenshot: Volatility failing on the XP memory image with NotImplementedError.

What the run above shows is a genuine failure, and it is worth understanding:

```
NotImplementedError: This version of Windows is not supported: 5.1 15.2600!
```

Both windows.netscan and windows.netstat fail identically on this image. determine_tcpip_version() in netscan.py only handles Vista and later, so it cannot walk the XP TCP/IP structures. There is no flag that fixes this, and no newer release that fixes it — the plugin was never written for XP.

What to do instead, and why the failure is still worth reporting:

Route	Works on XP?	Note
windows.netscan	No	NotImplementedError
windows.netstat	No	Same root cause
windows.handles	Partially	Can filter socket handles where supported
windows.dlllist	Yes	Shows Winsock loaded — corroborating, not conclusive
Correlate with the pcap	Yes	The strongest route when you have both

Report it as a method limitation, not a gap. "Network connections could not be enumerated from this XP image: Volatility 3's netscan does not support Windows 5.1. The pcap was used instead." That sentence is worth more than a silent omission, because it tells the reader exactly what was and was not examined.

This is the general lesson about memory analysis and old images. Volatility 3's XP support is real but incomplete. On a genuinely old case, check the plugin against the OS before you promise a result — and keep a Volatility 2 image to hand as a fallback.

6.6 What persists across a reboot

```
vol -f jo-2009.mem windows.svcscan
```

Services are how Windows malware usually survives a restart. svcscan lists every service, its binary path, and its start type:

```
vol -f jo-2009.mem windows.svcscan | head -30
vol -f jo-2009.mem windows.svcscan | grep -iE "SERVICE_AUTO_START" | head
```

Read the binary path, not the service name. Malware names services to look legitimate (Windows Update Helper) while pointing at a path that does not belong there — a user

© Billy Forrest @ https://training.forrest.one

directory, ProgramData, or a temp folder.

Automatic start services in a user-writable path are the finding. A service that starts itself at boot and runs from C:\Users\... is persistence, whatever it calls itself.

6.7 Memory artefacts worth knowing

Plugin	Answers
windows.dlllist	Which modules each process loaded — unexpected DLLs from temp paths
windows.filescan	Open file handles — files that existed but may not be in the MFT
windows.handles	Registry keys and other objects each process holds
windows.svcscan	Services and their start types
windows.registry.hivelist	Which registry hives were loaded
windows.hashdump	**Local account password hashes from memory**
windows.cachedump	Cached domain credentials

hashdump works from a memory image, and that is the point. Credentials sit in memory while a system runs. This is why a memory image of a domain controller is treated as catastrophic, and why you handle one with corresponding care.

7. Network evidence

A pcap shows what actually crossed the wire, which is the one thing logs can lie about.

```
capinfos 2025-01-22-traffic-analysis-exercise.pcap
```



Screenshot: capinfos summarising the capture, and DNS queries extracted from it.

What the run above shows. The capture's own metadata, before you read a single packet:

Field	Value	Reading
File type	pcap	Standard capture format
Encapsulation	Ethernet	Link-layer framing
Number of packets	39 k	A short but dense capture
File size	26 MB	
Capture duration	3202 s	About 53 minutes
Earliest packet	2025-01-22 19:44:56 UTC	
Latest packet	2025-01-22 20:38:18 UTC	

© Billy Forrest @ https://training.forrest.one

Field	Value	Reading
SHA256	e59db1c07c6f...	**The capture's integrity hash**

Start with capinfos, always. It tells you the time window you are working in, which is the first thing any timeline needs.

Find the suspicious traffic

```
# protocol hierarchy — what kinds of traffic are present
tshark -r capture.pcap -q -z io,phs

# every DNS query, unique
tshark -r capture.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name | sort -u

# filter to the interesting ones
... | grep -iE "authenticatoor|appointedtime"
```

What the run above shows. Two domains stand out immediately from the legitimate traffic:

Domain	Reading
authenticatoor.org	**A deliberately misspelt "authenticator"** — classic malvertising
appointedtimeagriculture.com	An unrelated-purpose domain, typical of a disposable C2 host

The spelling error is the finding. authenticatoor.org is designed to look right at a glance. Lookup domains character by character, not as words — typosquatting relies on your eye skipping the error.

Sort DNS by frequency as well as by name. C2 domains often appear once or twice, while legitimate domains appear hundreds of times. A domain queried exactly twice in 53 minutes, from a host that queried nothing else like it, is a beacon.

```
tshark -r capture.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name \
  | sort | uniq -c | sort -n | head -20     # rare names first
```

Extract what was transferred

```
# list HTTP objects, with the host each came from
tshark -r capture.pcap -Y "http.request" -T fields \
  -e frame.time -e ip.src -e http.host -e http.request.uri | head -20

# carve transferred files out of the capture
tshark -r capture.pcap --export-objects http,http_objects
ls http_objects/
```

--export-objects writes the actual transferred files to disk, which is how you go from "a file was downloaded" to holding the file. Hash everything you extract.

HTTPS hides the content, not the conversation. You still see the SNI hostname, the certificate, the timing and the volume. Beaconing is detectable from timing alone, even with no content, because a fixed interval with slight jitter is a machine, not a person.

The pcap question set

The bundled exercise asks six questions that map onto the analysis above. This is the shape of a real network investigation:

© Billy Forrest @ https://training.forrest.one

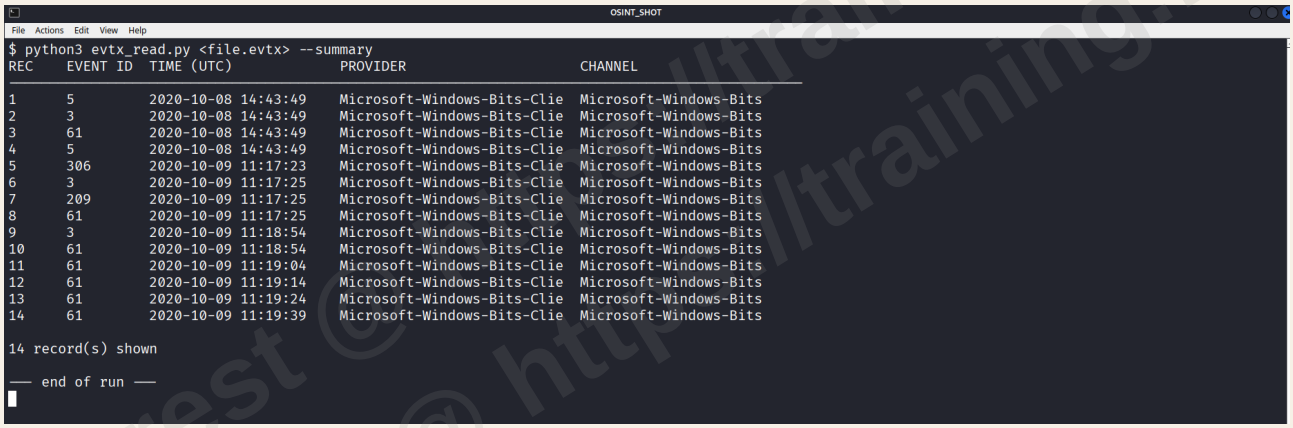
Question	Where the answer comes from
Which host is infected?	Who queries the odd domain first
Its MAC address?	eth.src for that IP, from DHCP
Its hostname?	DHCP hostname option, or NBNS
The user account?	Kerberos, NTLM, or cleartext auth
The fake download page?	The HTTP referer or DNS name
The C2 servers?	The hosts contacted after the download

Answer them in order, because each narrows the next. Identify the host, then filter everything else to that host.

8. Event logs

Windows records what it did. The EVTX files here are real attack data, organised by ATT&CK; tactic.

```
python3 evtx_read.py '<file>.evtx' --summary --limit 14
```



Screenshot: EVTX records summarised by event id, time and provider.

What the run above shows. Windows BITS (Background Intelligent Transfer Service) client events. BITS is a legitimate Windows component that malware abuses for downloading, because it is signed, always running, and its traffic blends in.

Rec	Event ID	Time	Provider	Reading
1	5	2020-10-08 14:43:49	BITS Client	Job created
2	3	2020-10-08 14:43:49	BITS Client	Job started
3	61	2020-10-08 14:43:49	BITS Client	Transfer started
5	306	2020-10-09 11:17:23	BITS Client	**Almost 21 hours later**
11–12	61	2020-10-09 11:19:04–14	BITS Client	**Repeated at 10-second intervals**

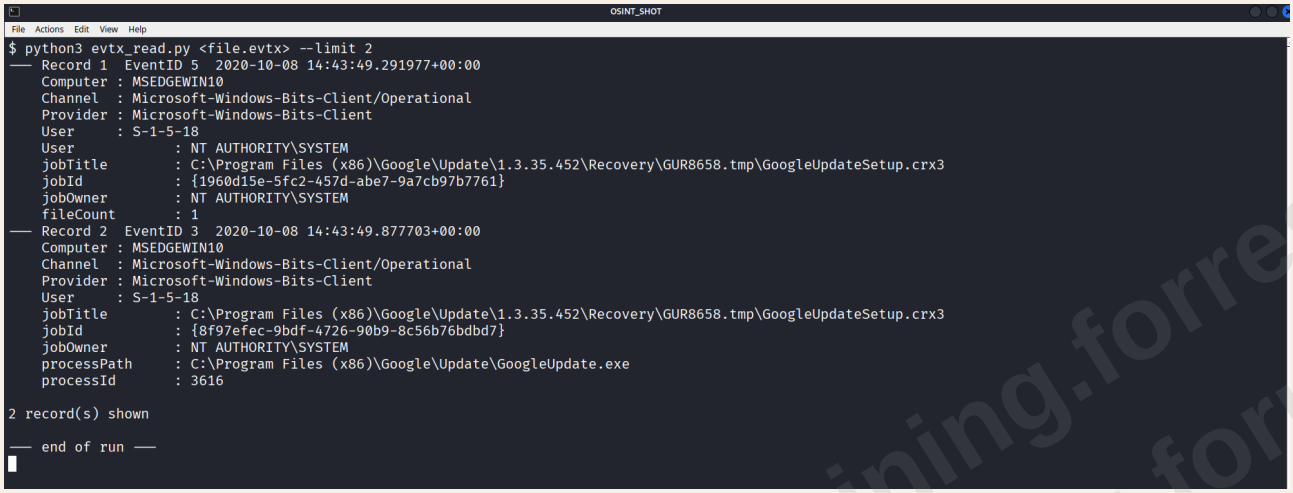
Rows 11 and 12 are the finding. Two transfer events ten seconds apart, from a service that should transfer files once, is a repeated transfer — which is what a malware download loop looks like.

© Billy Forrest @ https://training.forrest.one

Read the time column before the event id. The 21-hour gap between record 3 and record 5 matters more than any individual event id: it separates two distinct activity periods.

The detail view

```
python3 evtx_read.py '<file>.evtx' --limit 2
```



Screenshot: the full parsed record including event data fields.

What the run above shows. Every named field in the event — the part that matters, because EventData names are where the actual values live:

Field	Contains
Computer	Which machine logged it
Channel	Which log — Security, System, Sysmon, or a component log
Provider	Which Windows component wrote it
User	The security identifier of the actor
EventData	The named values: paths, PIDs, results, targets

The EventData names carry the meaning, not the numeric ids. Event ID 4688 means "a process was created" — the value is in the NewProcessName and CommandLine fields inside it. Always read the detail, never just the id.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Which log carries which signal

Event ID	Log	Means
4624	Security	A logon succeeded
4625	Security	**A logon failed — many of these is a brute force**
4672	Security	Special privileges were assigned
4688	Security	**A process was created** (needs auditing enabled)
4720	Security	A user account was created
7045	System	**A service was installed** — classic persistence
1	Sysmon	Process creation, with the full command line
3	Sysmon	A network connection was made
11	Sysmon	A file was created
22	Sysmon	A DNS query was made

Sysmon IDs are far more useful than Security IDs for detection, because Sysmon records the command line and the hash with the event. If 4688 has no command line, auditing is not configured to include it — and that is a finding about the environment, not about the attack.

Correlate across logs by time and PID. A 7045 service install at 14:43, a 1 process creation at 14:43, and a 3 network connection at 14:43 are the same event seen three ways. That triple is a detection you can deploy.

The EVTX tool problem, and the tool that replaces it

evtx_dump is installed but broken:

```
Traceback (most recent call last):
  File "/usr/local/bin/evtx_dump", line 3, in <module>
    from scripts.evtx_dump import main
ModuleNotFoundError: No module named 'scripts'
```

The cause is a packaging fault, not your command. The pipx console script imports a scripts package that is not present in the installed distribution. The library itself is fine.

The fix is to drive the library directly. tools/evtx_read.py in this guide does exactly that, and adds filtering by event id and a summary mode:

```
# summary of many records
python3 evtx_read.py file.evtx --summary --limit 50

# one specific event id
python3 evtx_read.py file.evtx --event-id 7045

# full detail for the first few
python3 evtx_read.py file.evtx --limit 5
```

Run it with the pipx venv's interpreter, because Evtx is installed there and not in the system Python:

```
/opt/pipx/venvs/python-evtx/bin/python evtx_read.py file.evtx --summary
```

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Keep the wrapper. When you hit a tool that is broken by its own packaging, fixing it once and keeping the fix is faster than re-diagnosing it on the next case — and the fix belongs in the report as a method note.

9. Reporting

An analysis that is not written up did not happen. The report is the deliverable, and it is the only part anyone else reads.

9.1 Structure

```
# Forensic Report — <case reference>

## Summary
Three sentences. What was examined, what was found, what it means.

## Evidence received
Item, description, serial or filename, source, date received.

## Integrity
SHA-256 on receipt, SHA-256 after analysis, and the statement that they match.

## Method
Every tool and version used, and every command run. Enough to reproduce it.

## Findings
One per finding, each with the artefact it came from and the confidence.

## Timeline
The chronological reconstruction, with the source of each event.

## Limitations
What could not be determined, with the reason. Include tool failures.

## Appendix
Full tool output, in the order it was produced.
```

The Integrity section is what makes the rest admissible. Put the two hashes next to each other so a reader can compare them without hunting:

```
Evidence received : ntfs1-gen1.E01
SHA-256 on receipt: 3f8a...c21d
SHA-256 after exam: 3f8a...c21d
Result            : MATCH — evidence unchanged
```

9.2 The findings table

#	Finding	Source	Confidence
1	OpenOffice spawned four shells in six seconds	pslist PIDs 3860, 2728, 3968, 1188	**High** — parent-child **confirmed**
2	soffice.bin held ten RWX memory regions	malfind PID 2392	**High** — corroborates finding 1
3	Three shell processes had unreadable command lines	cmdline PIDs 3860, 2728, 3968	**Medium** — consistent with clearing
4	Host queried authenticatoor.org	pcap DNS, 2025-01-22	**High** — direct observation
5	Network enumeration was not **possible** from this image	netscan NotImplementedError	**Limitation, stated**

© Billy Forrest @ https://training.forrest.one

Give every finding a confidence and a source. A finding with no source cannot be checked; a finding with no confidence cannot be weighed.

State the limitations as prominently as the findings. Finding 5 is not a failure of the analysis, it is a fact about the method, and the reader needs it to judge how much weight finding 2 can bear.

9.3 Two things that will be challenged

Any timestamp claim. Time zone, source of the clock, and whether the artefact even records that field. Windows A-time is often stale — say so, every time.

Any tool-dependent conclusion. malfind flags permissions, not intent, and false-positives on security software and JIT engines. If your finding rests on a heuristic, name the heuristic.

10. Tool reference

Tool	Phase	Command
file	Identify	file <evidence>
sha256sum	Preserve	sha256sum <evidence>
ssdeep	Identify	ssdeep -b <evidence>
ewfinfo	Acquire	ewfinfo <image>.E01
ewfexport	Acquire	ewfexport -q -t <dir> -f raw -u <image>.E01
dc3dd	Acquire	dc3dd if=/dev/sdX of=image.dd hash=sha256 log=acq.log
mmls	Disk	mmls <image>
fsstat	Disk	fsstat -o <offset> <image>
fls	Disk	fls -r -d <image>
istat	Disk	istat <image> <inode>
icat	Disk	icat <image> <inode> > out.file
blkls	Disk	blkls <image> > unallocated.bin
bulk_extractor	Carving	bulk_extractor -q -o out <image>
foremost	Carving	foremost -i <file> -o out
mactime	Timeline	mactime -b body.txt
log2timeline.py	Timeline	log2timeline.py --storage-file t.plaso <image>
psort.py	Timeline	psort.py -o l2tcsv -w t.csv t.plaso
vol	Memory	vol -f <image> windows.pslist
tshark	Network	tshark -r <pcap> -Y "<filter>"
capinfos	Network	capinfos <pcap>
evtx_read.py	Logs	python3 evtx_read.py <file>.evtx --summary

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Installing a missing tool

```
# apt — needs root. Names differ from the upstream project names:
#   libewf-utils and libpff-utils do NOT exist; the packages are ewf-tools and
#   afflib-tools. One wrong name aborts the whole apt transaction, so every
#   package in the list fails together.
apt-get install -y sleuthkit ewf-tools afflib-tools dc3dd dcfldd \
                   bulk-extractor foremost libesedb-utils

# Python tools — PEP-668 blocked, use pipx with an explicit bindir
export PIPX_HOME=/opt/pipx PIPX_BIN_DIR=/usr/local/bin
pipx install volatility3
pipx install plaso
```

Tools that are not in this lab, and why

Tool	Status	Note
ewfmount	**Fails**	No /dev/fuse in the container. Use ewfexport.
evtx_dump	**Broken**	Packaging fault. Use evtx_read.py.
netscan / netstat	**Fails on XP**	Plugin does not support Windows 5.1.
Autopsy	Not installed	GUI; needs a desktop session
hindsight	Not installed	Build fails on Python 3.14
python3-evtx	In pipx only	Call its venv interpreter explicitly

11. Troubleshooting

Every failure below is a real one hit while writing this guide.

Symptom	Cause	Fix
ewfmount: /dev/fuse not found	No FUSE in the container	ewfexport -t <dir> -f raw -u instead
ewfinfo: command not found	Package is ewf-tools, not libewf-utils	Install the right name
apt aborts and nothing installs	**One bad package name fails the whole transaction**	Remove the bad name, re-run
vol: no symbols / plugins return nothing	Symbol download failed offline	Pre-fetch with connectivity, or point -s at a cache
NotImplementedError: ... 5.1 15.2600	XP not supported by netscan	Report the limitation; use the pcap
FutureWarning: ... will be removed	malfind moved namespace	Use windows.malware.malfind
ModuleNotFoundError: No module named 'scripts'	evtx_dump packaging fault	Use evtx_read.py
ModuleNotFoundError: No module named 'Evtx'	Wrong interpreter	/opt/pipx/venvs/python-evtx/bin/python
log2timeline: unrecognized arguments: -o	plaso changed its CLI	Use --storage-file
mmls shows no partitions	Bare filesystem, no partition table	Correct — go straight to fsstat
file says data for an image	E01 is a container, not a filesystem	Expected — unwrap it first

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Symptom	Cause	Fix
fls returns nothing	Wrong offset, or encrypted volume	Check fsstat with -o
Carved files have no names	Carving has no filesystem context	Expected — state it in the report
filescan shows garbled characters	Pool-tag encoding on XP	Cosmetic; use the readable suffix

The apt lesson is worth stating on its own. apt-get install -y a b c d is a single transaction: if c does not exist, a, b and d are not installed either, and the only message is Unable to locate package c. On a fresh build this looks like apt being broken when it is behaving correctly. Install in small batches, and read the package list before running it.

12. Appendix — reproducing this guide

Every picture here is a capture of a real command run against real evidence. Nothing was reconstructed. The tooling is included so any figure can be regenerated.

```
# 1. Run the real command, keeping the transcript
docker exec -u 0 <lab> bash -lc \
  'export PATH=$PATH:/usr/local/bin; <command>' \
  > evidence/$(date -u +%Y%m%dT%H%M%SZ)_<slug>.txt 2>&1

# 2. Screenshot the terminal that produced it
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'

# 3. Crop the desktop chrome away so the text fills the page
tools/crop_shots.py images/*.png
```

The capture pitfalls the tool exists to handle, each of which cost real time:

- qterminal has no title flag; set the window title with a printf escape and find the window with xdotool search --class qterminal.
- Resize the window before the command runs. The shell wraps output to the width it had when it printed.
- qterminal rejects --size and then never launches.
- Use pkill -x qterminal, not -f: the -f form matches the docker exec command line and kills the caller.
- Never nest a single quote inside a command block passed to docker exec bash -lc inside a single-quoted string. It silently truncates the block, producing a garbled or empty capture rather than an error.

Where the evidence came from. All datasets are publicly published for education:

Dataset	Source
NPS test drive images	digitalcorpora.org/corpora/disk-images/
M57-Patents (disk, RAM, network)	digitalcorpora.org/corpora/scenarios/m57-patents-scenario/
Sleuth Kit test data	digitalcorpora.org/corpora/drives/tsk-2024/
EVTX attack samples	github.com/sbousseaden/EVTX-ATTACK-SAMPLES
Traffic analysis exercise	malware-traffic-analysis.net/2025/01/22/

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

The Digital Corpora bucket is readable without credentials:

```
curl -s "https://digitalcorpora.s3.amazonaws.com/?list-type=2&prefix=corpora/drives/"
aws s3 ls --no-sign-request s3://digitalcorpora/
```

What is in this folder

Path	Holds
artifacts/disk-images/	The disk, memory-adjacent, pcap and EVTX evidence
figures/	Excalidraw sources and rendered diagrams
images/	The terminal captures
evidence/	Raw command transcripts
tools/	Capture, crop, figure-build, EVTX parser and PDF renderer
reports/	Analysis reports produced from the evidence

Training at Forrest.one · Digital Forensics & Disk Image Analysis · Training evidence only.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one